Multi-Agent Mamba-RL for Efficient Sequential Recommendation

Zhi-Quan Feng

Abstract

Sequential recommendation is naturally connected to reinforcement learning because a recommender repeatedly observes user behavior, selects items, and receives feedback. In practice, however, reinforcement-learning recommenders face three persistent difficulties: user interests evolve at different time scales, logged datasets provide only partial counterfactual evidence, and large language or sequence backbones are expensive to execute inside every policy update. This report presents the independent Multi-Agent Mamba-RL pipeline implemented in `MediaTek_2026_project`. The pipeline fuses cached frozen Mamba-2.8B item-text embeddings with trainable two-layer LightGCN embeddings from the training-only user–item graph, then trains three lightweight agents: a long-term preference agent, a short-term intent agent, and a coordinator agent. Each agent owns a disjoint low-rank adaptation module. The two specialists use Mamba-inspired selective state-space recurrences with different observation horizons and decay time scales, while the coordinator learns user-dependent mixing weights and produces the final ranking policy.

Training follows a deliberately staged procedure: supervised specialist warm-up, frozen-specialist coordinator warm-up, and joint REINFORCE optimization. An optional full-catalog cross-entropy anchor reduces the mismatch between sampled-slate training and complete-catalog evaluation. At inference, the neural score can additionally be calibrated by popularity and first-order transition priors computed only from training histories; their coefficients are selected on validation data and default to zero for backward compatibility. The implementation supports MovieLens, Amazon Reviews 2023, and local Yelp 2019/2023 snapshots; performs chronological leave-two-out splitting; evaluates every user against the complete item catalog; records ranking and throughput metrics; and generates a grounded natural-language reason after recommendation. The architecture separates expensive semantic encoding, high-frequency policy learning, catalog ranking, calibration, and explanation generation. This separation makes the system suitable as a reproducible research baseline for studying multi-agent specialization, selective state dynamics, parameter-efficient policy learning, and high-throughput recommendation.

1 Introduction

Recommender systems have traditionally been formulated as rating prediction, click prediction, or next-item ranking problems. These formulations are effective when the objective is immediate relevance, but they do not fully represent the interactive nature of recommendation. A recommendation changes what the user sees, the resulting response changes the observable user state, and that new state influences subsequent recommendations. Reinforcement learning (RL) offers a natural language for this process by representing the recommender as a policy, the user as an environment, and engagement or satisfaction as a reward [3].

The promise of RL does not make its application straightforward. Industrial recommenders operate over thousands or millions of discrete actions, whereas most deep RL algorithms were developed for much smaller action spaces. Real user exploration is costly, so policies are usually trained from logged data generated by an earlier recommender. Feedback is incomplete: a missing click does not prove that an unseen item was irrelevant. A reward such as click-through rate may also favor immediate attraction even when the intended objective is long-term satisfaction, retention, or diversity. These issues make policy design, reward design, and offline evaluation inseparable parts of an RL recommender.

Sequential user representation creates another challenge. A user can exhibit a stable preference over months while simultaneously pursuing a short-lived intent within the current session. A single recurrent state must decide how quickly to forget old evidence, yet one global time scale is unlikely to fit both behaviors. Transformer recommenders can model long dependencies, but their self-attention cost grows quadratically with sequence length. Selective state-space models such as Mamba instead update a recurrent state in linear time and make the update input-dependent [2]. This mechanism is attractive for recommendation because an interaction can be selectively retained or forgotten according to its content.

A direct combination of a large Mamba language model and RL would nevertheless be computationally inefficient. If three agents each executed an independent 2.8-billion-parameter model for every sampled action, policy learning would be dominated by repeated text encoding rather than recommendation. Storing three complete adapted backbones would also undermine the parameter-efficiency motivation. The central systems question is therefore not simply whether Mamba can replace a Transformer, but how its semantic and selective-state capabilities can be used without placing a large foundation model inside the high-frequency optimization loop.

This project studies a fundamental question: *Can a shared Mamba semantic space and several low-rank, time-specialized policy agents provide an efficient and interpretable foundation for reinforcement-learning-based sequential recommendation?*

The proposed answer is a three-agent architecture. A frozen Mamba model first converts item metadata into a cached semantic matrix. A long-term agent and a short-term agent process sequences of these vectors with different selective state dynamics. A coordinator observes both specialist states, learns when each one should dominate, and produces the final catalog ranking. The agents are first initialized through supervised next-item learning and are then jointly optimized using policy gradients. This pipeline is implemented independently of the existing deep-learning and single-policy RL baselines, so it can be evaluated without changing their behavior.

2 Related Works and Innovations

2.1 Reinforcement Learning for Recommendation

Early deep RL recommenders commonly formulated recommendation as a Markov decision process and learned action values or policies from user feedback. DRN used deep Q-learning for news recommendation and explicitly modeled future reward and user return behavior [4]. List-wise and page-wise methods expanded the action from a single item to a slate, but this introduced a combinatorial action space. SlateQ addressed this problem by decomposing slate value under assumptions about user choice [6]. Policy-gradient methods offered another route: the Top-K off-policy REINFORCE recommender scaled policy learning to a production catalog and corrected bias from logged behavior policies [5].

These studies establish the reasons to use RL—exploration, delayed effects, and long-term value—but they also expose a gap between research formulations and common offline experiments. Many sequential recommenders use a sampled next-item RL objective even though the evaluation contains no environment transition caused by the recommended action. Recent work showed that part of the apparent benefit can arise because the RL term acts as an auxiliary representation-learning objective rather than because it learns a genuinely long-horizon policy [7]. A credible implementation must therefore distinguish its current offline objective from claims about online long-term satisfaction.

The present pipeline adopts a conservative interpretation. Its implemented reward is a sampled next-item reward, so it is best understood as a cooperative policy-gradient sequential ranker or contextual-bandit baseline. The code is structured so that the same agent interface can later receive simulator or online rewards, but the current MovieLens, Amazon, and Yelp evaluation does not claim to reconstruct unobserved counterfactual behavior.

2.2 Mamba and Sequential Recommendation

Transformer-based sequential recommenders capture rich dependencies but require quadratic attention computation. Mamba introduces input-dependent state-space parameters and linear sequence processing [2]. Mamba4Rec applied selective state-space modeling to sequential recommendation [8], while SIGMA introduced bidirectional and gated mechanisms to improve contextual and short-term modeling [9]. These results show that Mamba is useful for sequential recommendation, but replacing a Transformer with a standard Mamba block does not solve RL credit assignment, preference time-scale separation, or foundation-model cost.

This project uses Mamba in two distinct roles. The frozen Mamba-2.8B language model provides item semantics from natural-language metadata. Lightweight Mamba-inspired diagonal selective recurrences then provide trainable policy states. This separation preserves linear policy updates without executing the language model at every transition.

The three agents are therefore not three copies of Mamba-2.8B. They share one cached item-semantic space and own separate low-rank updates inside compact selective-state policies. Independent adaptation is retained while memory and computation remain proportional to the small recommendation dimension.

2.3 Multi-Agent and Multi-Time-Scale Recommendation

A multi-agent recommender should contain more than several neural modules. Each agent needs a distinct observation, policy, action distribution, reward signal, or functional role. In the proposed architecture, the long-term agent observes up to the full configured history, the short-term agent observes a recent window, and both define separate categorical policies. The coordinator is a third policy that observes specialist states and produces the deployed distribution.

This division reflects the fact that stable preferences and transient intents evolve at different rates. A user may consistently prefer one genre while temporarily searching within another category. A short-term specialist can react quickly, whereas a slowly changing specialist can resist noisy or accidental interactions. The coordinator performs context-dependent arbitration instead of using a fixed average.

LoRA represents an update with two low-rank matrices while leaving the base path unchanged [1]. Giving each agent its own LoRA parameters creates separate adaptation subspaces at a small cost. Zero-initializing the second matrix makes every adapter begin as a no-op and stabilizes staged training.

2.4 Why This Topic Must Be Investigated

Scientifically, the topic tests whether multi-agent specialization is more than extra parameter count. If long- and short-term agents receive identical inputs and objectives, they may collapse to nearly identical policies. A useful architecture must create structural differences through observation horizon, state time scale, and independent policy feedback, then verify these choices through ablation.

The topic also clarifies what Mamba contributes. A pretrained Mamba can supply textual knowledge, but repeatedly invoking it inside RL may erase any throughput advantage. A tiny recurrent model is efficient but may fail to exploit item metadata. Separating semantic precomputation from selective policy-state updates provides a testable middle ground.

Catalog ranking must also remain practical. Training over the full catalog at every step is expensive, while sampled-negative evaluation can exaggerate quality. This pipeline samples small slates for policy-gradient actions, optionally anchors supervised learning against the full catalog, and always evaluates by matrix multiplication against the complete catalog. It reports users per second and user–item scores per second so that efficiency is measurable.

Finally, explanation generation should not dominate ranking latency. The pipeline performs retrieval first and loads the generator only after evaluation. This creates a clear boundary between the policy and its post-hoc verbalization.

2.5 Innovations and Contributions

The implementation makes six contributions:

1. A cooperative three-agent recommender with structurally different long- and short-term selective-state specialists and a learned coordinator.
2. Disjoint LoRA parameters for every agent with shared cached Mamba semantics, avoiding three copies or forward passes of the 2.8B backbone.
3. Three-stage optimization separating specialist learning, coordinator calibration, and joint policy-gradient adaptation, with an optional full-catalog cross-entropy anchor.
4. Leakage-controlled catalog calibration combining the neural policy with training-only popularity and first-order transition statistics.
5. A unified MovieLens, Amazon Reviews 2023, and Yelp 2019/2023 evaluation interface with chronological splitting, periodic validation, best-checkpoint restoration, and full-catalog ranking.
6. Explanation generation isolated from ranking, with learned agent weights included in qualitative output.

3 Baselines

This section consolidates published MovieLens, Yelp, and Amazon results that are useful for positioning the proposed pipeline. The collection is protocol-aware. A dataset name is not a complete specification: papers apply different rating thresholds, k-core filters, sessionization rules, and chronological or random splits. Candidate construction is equally important. A score obtained by ranking one positive against sampled negatives is generally much higher than a score obtained by ranking the complete catalog. Consequently, the tables below are literature references, not a leaderboard across table boundaries.

The notation is $R@K$ for Recall@K, $H@K$ for Hit Ratio@K, $N@K$ for NDCG@K, and $M@K$ for MRR@K. With one held-out positive item per user, $R@K$ and $H@K$ are numerically equivalent, but each paper's notation is retained. *Main result* means every Amazon score reported for the paper's proposed method in its principal overall-result table; baseline rows copied into that paper are not duplicated unless the paper is itself a benchmark.

3.1 MovieLens-1M Sequential Recommendation

AMIM is a recent multi-interest sequential model evaluated on MovieLens-1M with Recall, NDCG, and MRR at 10 [**?**]. Its table is particularly relevant because it contains long-/short-interest and Transformer/RNN baselines. The scores below reproduce every MovieLens-1M row in the paper's main comparison; they must not be treated as directly comparable to this project's rating-≥ 4, chronological leave-two-out, full-catalog experiment without also reproducing AMIM's preprocessing and candidate protocol.

Table 1: Published MovieLens-1M results reported by AMIM (Expert Systems with Applications, 2024).

Method	$R@10$	$N@10$	$M@10$
BERT4Rec	.2043	.1053	.0735
GRU4Rec	.2065	.1013	.0712
STAMP	.2506	.1376	.1035
SASRec	.2632	.1411	.1088
SASRecD	.2705	.1443	.1106
MIND	.2401	.1355	.1021
SINE	.2365	.1275	.0965
TAMIC	.2684	.1425	.1112
AMIM	.2787	.1612	.1278

3.2 Yelp Results Under Distinct Protocols

Yelp dataset labels are especially overloaded. PAAC uses the standard implicit-feedback *Yelp2018* collaborative-filtering benchmark and reports complete-catalog top-20 metrics [**?**]. This is not the same corpus or task as this project's local Yelp 2019 rating-filtered sequential snapshot, but it provides a recent graph-recommendation reference that is relevant to the added LightGCN branch. Table **??** reproduces the conventional-test rows in PAAC's main comparison.

Table 2: Yelp2018 conventional-test results from PAAC (KDD 2024).

Method	$R@20$	$H@20$	$N@20$
LightGCN	.0591	.0614	.0483
Adapt-τ	.0724	.0753	.0603
SimGCL	.0720	.0748	.0594
PAAC	.0722	.0755	.0602

For an older but widely reused Yelp2018 reference point, the original LightGCN paper reports $R@20 = .0649$ and $N@20 = .0530$ under its own setup [**?**]. Differences between that row and PAAC's rerun illustrate why the implementation and split must accompany every score. Neither Yelp2018 table is used to claim improvement by the present Yelp19 experiment.

3.3 Amazon Reviews 2023: Full-Catalog and Generative Results

ReSID evaluates ten 5-core Amazon Reviews 2023 subsets with chronological leave-two-out splitting [10]. These are the closest broad, recent category-level references found, although ReSID is a Semantic-ID generative recommender rather than an RL policy.

Table 3: ReSID main results on Amazon Reviews 2023 5-core subsets.

Subset	$R@5$	$R@10$	$N@5$	$N@10$
Musical_Instruments	.0417	.0645	.0273	.0346
Video_Games	.0597	.0927	.0396	.0501
Industrial_and_Scientific	.0340	.0512	.0218	.0273

Subset	$R@5$	$R@10$	$N@5$	$N@10$
Baby_Products	.0285	.0441	.0186	.0236
Arts_Crafts_and_Sewing	.0313	.0485	.0205	.0261
Sports_and_Outdoors	.0210	.0323	.0137	.0174
Toys_and_Games	.0261	.0392	.0174	.0216
Health_and_Household	.0208	.0317	.0140	.0175
Beauty_and_Personal_Care	.0230	.0357	.0150	.0191
Books	.0530	.0737	.0369	.0436

IDIOMoE, accepted at ICLR 2026, reports full-catalog results for three large Amazon Reviews 2023 collections [11]. The source calls the first collection *Beauty(23)*; it is left under that exact name because the paper does not disambiguate it as `All_Beauty` or `Beauty_and_Personal_Care`. Table 2 includes all methods in the paper's large-catalog table.

Table 4: Full-catalog Amazon Reviews 2023 results from IDIOMoE.

	Beauty(23)		Books(23)		Toys(23)	
Method	$N@10$	$H@10$	$N@10$	$H@10$	$N@10$	$H@10$
SASRec	.0051	.0101	.0064	.0128	.0122	.0245
HSTU	.0130	.0247	.0211	.0410	.0149	.0332
ID Transformer	.0068	.0095	.0224	.0295	.0048	.0079
Text-Attr LLM	.0105	.0163	.0195	.0290	.0164	.0300
Item-LLM	.0082	.0119	.0174	.0261	.0079	.0148
IDIOMoE	.0119	.0228	.0224	.0419	.0186	.0361

BLaIR uses timestamp-based chronological splits on unfiltered Amazon Reviews 2023 and evaluates UniSRec with different semantic encoders [12]. It therefore has much larger catalogs than this project's current All Beauty 5-core preprocessing. Table 3 reproduces all sequential-recommendation $N@10$ scores in its main encoder comparison.

Table 5: BLaIR/UniSRec $N@10$ on unfiltered Amazon Reviews 2023.

Semantic encoder	All_Beauty	Video_Games	Baby_Products
RoBERTa-large	.0177	.0113	.0070
Qwen3-Embedding-0.6B	.0224	.0126	.0072
Sup-SimCSE-RoBERTa-large	.0232	.0114	.0069
Sentence-T5-large	.0212	.0125	.0075
Qwen3-Embedding-4B	.0212	.0127	.0075
Qwen3-Embedding-8B	.0216	.0138	.0072
SFR-Embedding-Mistral	.0231	.0131	.0076
E5-Mistral-7B-Instruct	.0238	.0136	.0072
GritLM-7B	.0216	.0133	.0074
Gemini-Embedding-001	.0241	.0136	.0073
Text-Embedding-3-Large	.0237	.0135	.0078

3.4 Amazon Reviews 2018 and Legacy Sequential Benchmarks

MARS uses Amazon Reviews 2018 5-core sequences and chronological leave-one-out evaluation [13]. *Prime Pantry* is retained as reported and is not silently relabeled as `Grocery_and_Gourmet_Food`.

Table 6: MARS main results on Amazon Reviews 2018 5-core subsets.

Subset	$R@10$	$N@10$
Industrial_and_Scientific	.1533	.1086
Prime Pantry	.0928	.0591
Musical_Instruments	.1082	.0846
Arts_Crafts_and_Sewing	.1684	.1306
Office_Products	.1571	.1270

Table 5 adds recent Mamba recommenders and established sequential references. SIGMA is an AAAI 2025 Mamba model [9]; HM2Rec is an AAAI 2026 hyperbolic graph–Mamba model with a mixture-of-experts path [14]; MQSA-TED is a WSDM 2024 transition-distillation model [15]; and Mamba4Rec, DuoRec, and FMLP-Rec provide established references [8, 16, 17]. Dashes denote unreported metrics. HM2Rec's high values come from sampled-candidate evaluation and must not be compared numerically with full-catalog rows.

Table 7: Legacy Amazon sequential results under each paper's own protocol.

Model	Subset	$H@5$	$N@5$	$H@10$	$N@10$	$M@10$	$H@20/N@20$
SIGMA	Beauty	–	–	.0986	.0604	.0488	–
SIGMA	Sports	–	–	.0735	.0590	.0556	–
SIGMA	Games	–	–	.1627	.1088	.0924	–
HM2Rec	Books	–	–	.7874	.6053	–	.8487/.6209
HM2Rec	Toys	–	–	.6295	.4383	–	.7262/.4629
MQSA-TED	Beauty	.0752	.0534	.1039	.0627	–	.1435/.0726
MQSA-TED	Sports	.0455	.0320	.0643	.0380	–	.0906/.0446
MQSA-TED	Toys	.0834	.0600	.1130	.0696	–	.1503/.0789
Mamba4Rec	Beauty	–	–	.0812	.0451	.0362	–
Mamba4Rec	Video Games	–	–	.1152	.0603	.0438	–
DuoRec	Beauty	–	–	.0845	.0443	–	–
DuoRec	Clothing	–	–	.0302	.0148	–	–
FMLP-Rec	Beauty	.0398	.0258	.0632	.0333	–	.0958/.0415
FMLP-Rec	Sports	.0218	.0144	.0344	.0185	–	.0537/.0233
FMLP-Rec	Toys	.0456	.0317	.0683	.0391	–	.0991/.0468

3.5 Different Amazon Recommendation Tasks

The following results are useful references but are not sequential next-item results. JGCF performs user–item collaborative filtering on Amazon Books [18]. The Amazon Fashion study formulates recommendation as dense retrieval from a review query to a purchased product in an 826,402-item catalog [19].

Table 8: Published Amazon scores for non-sequential tasks.

Task/model	Subset	$R@10$	$N@10$	$R@20$	$N@20$	$R@50$	$N@50$
CF/JGCF	Books	.0796	.0671	.1191	.0798	.1949	.1019

Dense-retrieval model	Subset	$R@10$	$M@10$
BM25	Amazon_Fashion	.2600	.2500
Zero-shot MPNet	Amazon_Fashion	.4200	.3600
Fine-tuned MPNet	Amazon_Fashion	.6600	.5900

3.6 Coverage of the Requested Amazon Subsets

Table 7 audits every requested subset. *Not found* means that no verified category-specific score was found in the selected primary papers above; it does not prove that no result exists anywhere. Legacy labels such as *Beauty*, *Games*, or *Toys* are not automatically mapped to Amazon Reviews 2023 categories.

Table 9: Coverage audit for all requested Amazon subsets.

Requested subset	Verified source and comparability
All_Beauty	BLaIR, Amazon Reviews 2023, unfiltered full-catalog $N@10$
Amazon_Fashion	Dense retrieval only; not sequential recommendation
Appliances	Not found in the selected primary papers
Arts_Crafts_and_Sewing	ReSID (2023 5-core); MARS (2018 5-core)
Automotive	Not found in the selected primary papers
Baby_Products	ReSID (2023 5-core); BLaIR (2023 unfiltered)
Beauty_and_Personal_Care	ReSID (2023 5-core); IDIOMoE's Beauty(23) kept separate
Books	ReSID and IDIOMoE (2023); JGCF/HM2Rec legacy protocols
Cell_Phones_and_Accessories	Not found in the selected primary papers
Clothing_Shoes_and_Jewelry	DuoRec reports legacy Clothing only
Digital_Music	Not found in the selected primary papers
Electronics	Not found in the selected primary papers
Gift_Cards	Not found in the selected primary papers
Grocery_and_Gourmet_Food	Not found; MARS Prime Pantry is not equivalent
Handmade_Products	Not found in the selected primary papers
Health_and_Household	ReSID, Amazon Reviews 2023 5-core
Health_and_Personal_Care	Not found in the selected primary papers
Home_and_Kitchen	Not found in the selected primary papers
Industrial_and_Scientific	ReSID (2023 5-core); MARS (2018 5-core)
Kindle_Store	Not found in the selected primary papers
Magazine_Subscriptions	Not found in the selected primary papers
Movies_and_TV	Not found in the selected primary papers
Musical_Instruments	ReSID (2023); MARS legacy protocol
Office_Products	MARS, Amazon Reviews 2018 5-core

Requested subset	Verified source and comparability
Patio_Lawn_and_Garden	Not found in the selected primary papers
Pet_Supplies	Not found in the selected primary papers
Software	Not found in the selected primary papers
Sports_and_Outdoors	ReSID (2023); SIGMA/MQSA-TED/FMLP-Rec legacy Sports
Subscription_Boxes	Not found in the selected primary papers
Tools_and_Home_Improvement	Not found in the selected primary papers
Toys_and_Games	ReSID and IDIOMoE (2023); HM2Rec/MQSA-TED/FMLP-Rec legacy Toys
Video_Games	ReSID and BLaIR (2023); Mamba4Rec legacy Video Games
Unknown	Not a standard benchmark subset; no verified score

3.7 Comparison Rule for This Project

The present All Beauty experiment uses a project-specific 5-core catalog with 1,620 users and 6,982 items and performs full-catalog ranking. Its $R@10 = .15432$ therefore must not be described as outperforming BLaIR, BERT4Rec, HM2Rec, or another paper solely because the number is larger. A defensible comparison requires the same raw Amazon release, category mapping, filtering, split, seen-item masking, candidate catalog, and metric code. The recommended reporting order is: (1) reproduce SASRec and Mamba4Rec in this project's data loader; (2) report the proposed model and those baselines under the identical full-catalog evaluator; and (3) retain the literature tables in this section only as external reference points.

4 Method

4.1 Problem Formulation

Let $\mathcal{U}$ be users and $\mathcal{I}$ the catalog, with $N = |\mathcal{I}|$. User interactions are ordered:

$$\mathcal{H}_u = (i_1, i_2, \ldots, i_{T_u}). \tag{1}$$

At transition t, the state is prefix $s_t = (i_1, \ldots, i_t)$ and logged next item i_{t+1} is positive. The policies are

$$\pi_L(a \mid s_L), \qquad \pi_S(a \mid s_S), \qquad \pi_C(a \mid s_L, s_S). \tag{2}$$

4.2 Pipeline Overview

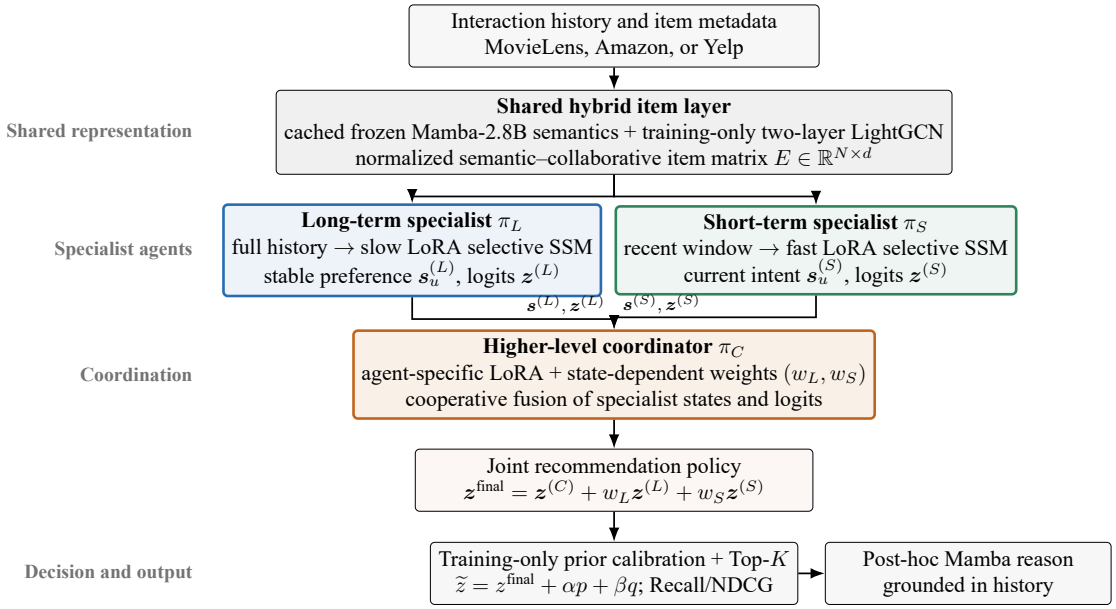

Figure 1: Hierarchical cooperative Multi-Agent Mamba-RL architecture. Solid arrows show the inference path; dashed red arrows show training-only policy feedback.

The hierarchy assigns complementary responsibilities rather than making three agents vote independently. The two specialist policies extract preferences at different time scales, and the higher-level coordinator decides their state-dependent contribution to one deployed recommendation policy. During joint training, the coordinator and both specialists receive policy-gradient signals; during inference, only the solid forward path is executed.

4.3 Data and Chronological Splitting

Every record becomes (u, i, τ, t_i): user, item, timestamp, and item text. IDs become contiguous integers. Users need at least five positive events by default. MovieLens and Yelp use rating at least four.

Table 10: Dataset interfaces and item text.

Argument	Interactions	Item text
movielens	ratings	title and genres
amazon-<category>	Amazon Reviews 2023	title, category, features, description
amazon:<exact-config>	exact HF config	review/meta join by `parent_asin`
yelp19, yelp23	local Yelp JSON/table	name, categories, location, attributes
synthetic	deterministic events	short names for tests

Yelp is license-gated and requires a local path. Leave-two-out splitting is

$$\mathcal{H}_u^{\text{train}} = (i_1, \ldots, i_{T_u-2}), \tag{3}$$

$$i_u^{\text{valid}} = i_{T_u-1}, \tag{4}$$

$$i_u^{\text{test}} = i_{T_u}. \tag{5}$$

Training prefixes produce $\big((i_1, \ldots, i_k), i_{k+1}\big)$. Reservoir sampling caps transitions at `MAX_TRANSITIONS`.

4.4 Cached Mamba Semantics

Frozen Mamba-2.8B encodes metadata:

$$\boldsymbol{e}_i^{\text{text}} = \text{MeanPool}\left(\text{Mamba}_{\text{frozen}}(t_i)\right) \in \mathbb{R}^{d_m}. \tag{6}$$

A dataset/item/text fingerprint controls cache reuse. A shared bridge gives

$$\boldsymbol{e}_i = \text{norm}(W_p \boldsymbol{e}_i^{\text{text}}), \qquad W_p \in \mathbb{R}^{d \times d_m}, \quad d = 128. \tag{7}$$

4.5 Training-Only LightGCN Fusion

The current implementation enables a two-layer LightGCN branch by default [**?**]. It constructs an undirected bipartite graph only from unique user–item pairs in `train_by_user`; validation and test edges are excluded. Let A be its adjacency matrix, D its degree matrix, and $X^{(0)} \in \mathbb{R}^{(|\mathcal{U}|+|\mathcal{I}|)\times d}$ trainable node embeddings. The code performs symmetric normalized propagation without feature transforms or nonlinearities:

$$X^{(\ell+1)} = D^{-1/2} A D^{-1/2} X^{(\ell)}, \qquad \ell \in \{0, 1\}, \tag{8}$$

$$\bar{X} = \frac{1}{3} \sum_{\ell=0}^{2} X^{(\ell)}. \tag{9}$$

Writing $\boldsymbol{e}_i^{\text{gcn}}$ for item i's row of $\bar{X}$, the actual vector consumed by both specialists and by catalog scoring is

$$\boldsymbol{e}_i = operatornamenorm\big(\text{norm}(W_p \boldsymbol{e}_i^{\text{text}}) + \text{norm}(\boldsymbol{e}_i^{\text{gcn}})\big). \tag{10}$$

Thus the graph is a shared collaborative item encoder, not a fourth policy agent. It is optimized with the item projection during specialist warm-up and joint training, frozen during coordinator warm-up, and recomputed once per batch for reuse by histories and candidates. The optional `--graph-device` places graph parameters and edges on a second GPU; `--no-use-graph-embeddings` provides the semantic-only ablation.

4.6 Agent-Specific LoRA

For input $\boldsymbol{x}$, agent j owns

$$\text{LoRA}_j(\boldsymbol{x}) = \frac{\alpha}{r} B_j A_j \,\text{Dropout}(\boldsymbol{x}). \tag{11}$$

Defaults are $r = 8$, $\alpha = 16$, and dropout 0.05. A_j uses Kaiming initialization and B_j starts at zero. Specialists own candidate, delta, gate, and output adapters; the coordinator owns state, mixture, and score adapters.

4.7 Selective-State Specialists

For $j \in \{L, S\}$:

$$\boldsymbol{\delta}_t^{(j)} = \text{softplus}\left(\boldsymbol{b}_\delta^{(j)} + \text{LoRA}_\delta^{(j)}(\boldsymbol{x}_t)\right), \tag{12}$$

$$\boldsymbol{a}_t^{(j)} = \exp(-\boldsymbol{\delta}_t^{(j)}), \tag{13}$$

$$\widetilde{\boldsymbol{h}}_t^{(j)} = \tanh\left(\boldsymbol{x}_t + \text{LoRA}_{\text{candidate}}^{(j)}(\boldsymbol{x}_t)\right), \tag{14}$$

$$\boldsymbol{h}_t^{(j)} = \boldsymbol{a}_t^{(j)} \odot \boldsymbol{h}_{t-1}^{(j)} + (1 - \boldsymbol{a}_t^{(j)}) \odot \widetilde{\boldsymbol{h}}_t^{(j)}, \tag{15}$$

$$\boldsymbol{g}_t^{(j)} = \text{sigmoid}\left(\text{LoRA}_{\text{gate}}^{(j)}(\boldsymbol{x}_t)\right), \tag{16}$$

$$\boldsymbol{o}_t^{(j)} = \boldsymbol{g}_t^{(j)} \odot \boldsymbol{h}_t^{(j)} + (1 - \boldsymbol{g}_t^{(j)}) \odot \boldsymbol{x}_t. \tag{17}$$

Small $\boldsymbol{\delta}_t$ preserves history; large values adopt the current item. Long sees up to 100 events at initial time scale 0.08; short sees 10 at 0.5. Logits are

$$z_i^{(j)} = \exp(\tau_j)\langle \boldsymbol{s}_u^{(j)}, \boldsymbol{e}_i \rangle. \tag{18}$$

4.8 Coordinator

$$\boldsymbol{q}_u = [\boldsymbol{s}_u^{(L)}; \boldsymbol{s}_u^{(S)}], \tag{19}$$

$$[w_L, w_S] = \text{softmax}(\text{LoRA}_{\text{mix}}(\boldsymbol{q}_u)), \tag{20}$$

$$\boldsymbol{s}_u^{(C)} = \text{norm}\left(w_L \boldsymbol{s}_u^{(L)} + w_S \boldsymbol{s}_u^{(S)} + \text{LoRA}_{\text{state}}(\boldsymbol{q}_u)\right), \tag{21}$$

$$z_i^{\text{final}} = z_i^{(C)} + w_L z_i^{(L)} + w_S z_i^{(S)}. \tag{22}$$

4.9 Training-Only Catalog Calibration

The neural coordinator can be augmented by two deterministic catalog priors. They are estimated strictly from $\mathcal{H}_u^{\text{train}}$ and therefore introduce no validation or test interactions into the feature construction. Let c_i be the number of occurrences of item i in all training histories. The standardized log-popularity feature is

$$p_i = \frac{\log(1 + c_i) - \mu_p}{\max(\sigma_p, 10^{-6})}. \tag{23}$$

For adjacent training events, let $n_{a,b}$ count transitions from item a to item b and define

$$q_{a,b} = \log(1 + n_{a,b}). \tag{24}$$

If ℓ_u is the last observed item in the inference history, the deployed full-catalog score is

$$\widetilde{z}_{u,i} = z_{u,i}^{\text{final}} + \alpha p_i + \beta q_{\ell_u, i}, \qquad q_{\ell_u,i} = 0 \text{ for unseen transitions.} \tag{25}$$

Seen non-target items are masked after calibration. The coefficients α and β are selected using validation ranking only; the held-out test set is evaluated once after the choice is fixed. A negative α suppresses over-popular items, while a positive β reinforces locally plausible next-item transitions. This module is not a fourth RL agent and does not update model weights. Both coefficients default to zero, so existing runs retain the original neural ranking exactly.

5 Training Strategy

5.1 Stage 1: Specialist Warm-Up

A sampled slate is $\mathcal{C} = \{i^+, i_1^-, \ldots, i_{K-1}^-\}$, where K is configurable and defaults to 64. The coordinator is frozen:

$$\mathcal{L}_{\text{specialist}} = \text{CE}(\boldsymbol{z}^{(L)}, y) + \text{CE}(\boldsymbol{z}^{(S)}, y). \tag{26}$$

In sampled mode, $y = 0$ because the positive item is placed first. With `--full-catalog-supervised`, both specialist logit vectors contain all N items and $y = i^+$. This option removes negative-sampling noise during the warm-up at the cost of a larger matrix multiplication.

5.2 Stage 2: Coordinator Warm-Up

Specialists and projection are frozen:

$$\mathcal{L}_{\text{coord}} = \text{CE}(\boldsymbol{z}^{\text{final}}, y). \tag{27}$$

The target convention follows Stage 1: $y = 0$ for a sampled slate and $y = i^+$ for full-catalog supervision.

5.3 Stage 3: Joint REINFORCE

Each policy forms $\pi_j(a \mid s) = \text{softmax}(\boldsymbol{z}^{(j)})_a$ and samples an action:

$$r_j = \begin{cases} 1, & a_j = 0, \\ -0.05, & a_j \neq 0. \end{cases} \tag{28}$$

Baselines use $b_j \leftarrow 0.95 b_j + 0.05 \overline{r}_j$ and $A_j = r_j - b_j$. The policy loss is

$$\mathcal{L}_{\text{PG}} = -\sum_j \mathbb{E}[A_j \log \pi_j(a_j \mid s)] - 0.01 \sum_j \mathbb{E}[H(\pi_j)]. \tag{29}$$

Advantage is detached. Additional terms are

$$\mathcal{L}_{\text{sup}} = \frac{1}{3} \sum_{j \in \{L,S,C\}} \text{CE}(\boldsymbol{z}^{(j)}, y), \tag{30}$$

$$\mathcal{L}_{\text{spec}} = \cos^2(\boldsymbol{s}_u^{(L)}, \boldsymbol{s}_u^{(S)}), \tag{31}$$

$$\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{PG}} + \lambda_{\text{sup}} \mathcal{L}_{\text{sup}} + \lambda_{\text{spec}} \mathcal{L}_{\text{spec}}. \tag{32}$$

The defaults are $\lambda_{\text{sup}} = 0.1$ and $\lambda_{\text{spec}} = 0.01$. Joint REINFORCE actions are still sampled from the configured slate. When full-catalog supervision is enabled, a second catalog projection computes $\mathcal{L}_{\text{sup}}$ over all N items; otherwise the auxiliary cross-entropy uses the sampled slate. Thus the policy gradient remains inexpensive while the supervised anchor can be aligned with the evaluation action space.

5.4 Gradients and Optimization

Coordinator gradients flow through fusion into both specialist LoRA paths. Each specialist also receives its own policy loss. This is cooperative local credit rather than a centralized counterfactual critic.

Table 11: Default optimization.

Stage	Learning rate	Trainable components
Specialists	2×10^{-4}	long, short, projection
Coordinator	2×10^{-4}	coordinator
Joint RL	5×10^{-5}	all components

AdamW uses weight decay 10^{-5}. CUDA uses FP16 autocast, GradScaler, and gradient clipping at 1.0. Validation runs every 1,000 optimizer steps by default. A `ReduceLROnPlateau` scheduler halves the coordinator or joint learning rate after a configurable number of non-improving checks, and joint training supports early stopping. Each improvement is atomically saved as `best_validation.pt`; final metrics are computed only after restoring that validation-selected state.

6 Full-Catalog Evaluation and Throughput

Evaluation always projects the complete catalog once, irrespective of whether training used sampled or full-catalog supervision:

$$E = W_p E_{\text{text}} \in \mathbb{R}^{N \times d}, \qquad Z^{\text{neural}} = S E^\top \in \mathbb{R}^{B \times N}. \tag{33}$$

This GEMM avoids a $B \times N \times d$ expansion. Optional popularity and transition vectors are then added by broadcasting and sparse row updates to obtain $\widetilde{Z}$; they do not require another model forward pass. Seen non-target items receive $-\infty$.

$$\text{rank}_u = \sum_{i=1}^{N} \mathbb{I}[\widetilde{z}_{u,i} \geq \widetilde{z}_{u,i^+}], \tag{34}$$

$$\text{Recall@K} = \frac{1}{|\mathcal{U}|} \sum_{u} \mathbb{I}[\text{rank}_u \leq K], \tag{35}$$

$$\text{NDCG@K} = \frac{1}{|\mathcal{U}|} \sum_{u} \frac{\mathbb{I}[\text{rank}_u \leq K]}{\log_2(\text{rank}_u + 1)}. \tag{36}$$

The code reports $K = 5, 10$, users/s, and scores/s.

Throughput comes from cached Mamba vectors, shared semantics, linear recurrences, capped history, sampled RL action slates, optional batched full-catalog CE, FP16, bounded transitions, GEMM ranking, sparse prior updates, and delayed generator loading.

7 Recommendation Explanation

For sampled test users, Top-1, ten recent item texts, and coordinator weights form:

```
Explain this recommendation in one concise sentence using only the supplied
    history.
Long-term agent weight=<wL>; short-term agent weight=<wS>.
History: <recent item texts>.
Recommended item: <recommended item text>. Reason:
```

Mamba greedily generates at most 40 tokens after ranking. The explanation is grounded but post-hoc; it is not a causal proof and does not affect reward.

8 Experimental Design

Required ablations include single versus three agents, identical versus 100/10 windows, fixed versus learned fusion, direct joint RL versus staged training, sampled versus full-catalog supervised anchoring, and removal of specialization. Catalog calibration must be ablated as neural-only, popularity-only, transition-only, and combined scoring, with α and β selected on validation for every dataset. Semantic features should compare Mamba vectors, random vectors, and ID embeddings. Sequence baselines should include GRU and Transformer under matched capacity. Random negatives should be compared with hard negatives.

Beyond Recall/NDCG, useful measures include coverage, novelty, diversity, results by history length, coordinator-weight distribution, specialist-state cosine similarity, transitions/s, users/s, scores/s, peak GPU memory, and explanation latency. Because the priors can change exposure concentration, coverage and popularity-stratified Recall should accompany aggregate Recall.

8.1 Amazon All Beauty Iterative Study

The implemented changes were tested on the Amazon Reviews 2023 All Beauty 5-core split with 1,620 users, 6,982 items, 11,744 training interactions, chronological leave-two-out splitting, and full-catalog evaluation. Table 10 reports the held-out results recorded by the training pipeline.

Table 12: Iterative Amazon All Beauty results. Rounds are an engineering sequence rather than independent causal ablations.

Round	Main change	Valid Recall@10	Test Recall@10
1	Sampled-slate baseline	0.11173	0.13519
2	Full-catalog CE warm-up and joint CE anchor	0.11481	0.13827
3	Continued joint RL from the Round-2 checkpoint	0.11605	0.13395
4	Frozen Round-2 neural checkpoint plus training-only calibrated priors	0.13086	**0.15432**

Round 2 changes the supervised action space and Round 4 changes the deployed scoring rule, so these are algorithmic modifications rather than only different hyperparameters. For Round 4, validation selected $\alpha = -0.25$ and $\beta = 4.0$. The neural checkpoint was not updated, and the test set remained held out until the coefficients were fixed. The resulting Test Recall@10 of 0.15432 exceeded the predefined target of 0.15. This comparison does not by itself isolate causal contributions; the prior components and the full-catalog anchor should still be reported as separate controlled ablations.

8.2 MovieLens-1M and Yelp19 Held-Out Results

Table **??** records the selected best-validation checkpoints already produced by this repository. Both runs use rating ≥ 4, chronological leave-two-out, one test target per user, and full-catalog ranking; therefore Recall and Hit Ratio coincide. LightGCN is enabled and its edges, as well as the popularity and transition calibration statistics, come only from training histories. These rows are directly comparable to each other only at a metric-definition level: the catalogs and calibrated coefficients differ.

Table 13: Current project full-catalog test results.

Dataset	Run	$R@5$	$R@10$	$N@5$	$N@10$
MovieLens-1M	20260827_011510	.14899	.21793	.10257	.12476
Yelp19	20260827_030443	.06940	.09685	.04656	.05545

The published tables in Section 3 remain contextual references rather than evidence of superiority because AMIM and Yelp2018 use different preprocessing and, in Yelp's case, a different corpus and cutoff.

9 Implementation and Reproducibility

Table 14: Independent implementation files.

File	Responsibility
`run_mamba_rl.sh`	launcher
`src/data_mamba_rl.py`	dataset adapters
`src/model_mamba_rl.py`	agents and LoRA
`src/train_mamba_rl.py`	training, evaluation, explanations
`tests/test_mamba_rl.py`	tests
`outputs_mamba_rl/`	artifacts

```
bash run_mamba_rl.sh movielens-1m 0
bash run_mamba_rl.sh movielens-20m 0
bash run_mamba_rl.sh amazon-all-beauty 0
bash run_mamba_rl.sh amazon-movies-and-tv 0
bash run_mamba_rl.sh yelp19 0 /data/yelp-2019
bash run_mamba_rl.sh yelp23 0 /data/yelp-2023
```

Environment variables include CACHE_DIR, BATCH_SIZE, EVAL_BATCH_SIZE, SPECIALIST_EPOCHS, COORDINATOR_EPOCHS, RL_EPOCHS, VALIDATE_EVERY_STEPS, MONITOR_METRIC, EARLY_STOPPING_P LR_PATIENCE, MAX_TRANSITIONS, SEED, POPULARITY_ALPHA, TRANSITION_BETA, USE_GRAPH_EMBEDD GRAPH_DEVICE, TARGET_RECALL_AT_10, EXPERIMENT_NOTE, and RESUME_CHECKPOINT. The Round-4 evaluation can be reproduced with

```
SPECIALIST_EPOCHS=0 COORDINATOR_EPOCHS=0 RL_EPOCHS=0 \
POPULARITY_ALPHA=-0.25 TRANSITION_BETA=4.0 \
MONITOR_METRIC=recall@10 \
RESUME_CHECKPOINT=outputs_mamba_rl/amazon-all-beauty/\
20260826_155853/multi_agent_lora.pt \
bash run_mamba_rl.sh amazon-all-beauty 0
```

A CPU smoke test is

```
python -m src.train_mamba_rl \
  --dataset synthetic --device cpu --skip-mamba \
  --no-generate-reasons --max-transitions 32 --batch-size 8
```

Outputs are train_<run-id>.log, best_validation.pt, multi_agent_lora.pt, loss.png, metrics.json, and recommendations.json under outputs_mamba_rl/<dataset>/<run- Every log begins with EXPERIMENT_NOTE and the complete serialized EXPERIMENT_CONFIG; calibrated runs additionally record CATALOG_PRIORS, vector provenance, resumed checkpoint, VALID_BEST, TEST_FINAL, and TARGET_RESULT. Legacy training paths do not import this pipeline.

10 Limitations

1. The $+1/-0.05$ next-item reward is not online feedback from the learned policy.
2. Static logs contain no action-dependent user transition, so this is not a complete long-horizon MDP.
3. Random negatives may be unexposed; no propensity correction is used.
4. Credit assignment uses local rewards rather than a centralized critic.
5. REINFORCE still samples slates; full-catalog CE reduces but does not remove the policy-gradient/evaluation mismatch.
6. Popularity and first-order transition counts are simple, dataset-dependent priors and may not transfer across domains or temporal shifts.
7. Calibrated gains must be separated from gains in the learned neural policy; the priors are deterministic post-training score corrections.
8. Explanations are post-hoc and not optimized for faithfulness.
9. Mamba-2.8B supplies semantics; agents use compact Mamba-inspired recurrences rather than three full 2.8B adapters.

Recall/NDCG gains therefore mean improved offline ranking, not proven long-term satisfaction.

11 Future Research

Reward-adaptive memory can use

$$\boldsymbol{\delta}_t = f_\theta(\boldsymbol{x}_t, r_{t-1}, \sigma_{t-1}). \tag{37}$$

Offline RL can add propensity weighting or

$$\mathcal{L}_{\text{CQL}} = \log \sum_a \exp Q(s,a) - Q(s, a_{\text{logged}}). \tag{38}$$

A simulator can support centralized critics and retention reward. Item selection and reasoning can be jointly optimized with

$$r = \lambda_1 r_{\text{rank}} + \lambda_2 r_{\text{grounding}} + \lambda_3 r_{\text{consistency}} + \lambda_4 r_{\text{diversity}}. \tag{39}$$

12 Conclusion

The pipeline combines Mamba item semantics, linear selective-state modeling, agent-specific LoRA, and policy gradients without placing three large models in the RL loop. Mamba-2.8B establishes a reusable semantic catalog; long and short agents model different time scales; and a coordinator learns context-dependent fusion. Staged training establishes specialists, calibrates coordination, and performs joint adaptation, while optional full-catalog CE aligns the supervised anchor with the evaluation action space. Training-only popularity and transition priors provide a leakage-controlled calibration layer whose effect remains explicitly separable from learned model quality. Periodic validation, best-checkpoint restoration, auditable experiment logs, full-catalog evaluation, and isolated explanation generation make ranking quality and systems cost measurable.

References

[1] E. Hu, Y. Shen, P. Wallis, et al. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.

[2] A. Gu and T. Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *COLM*, 2024.

[3] M. M. Afsar, T. Crump, and B. Far. Reinforcement Learning Based Recommender Systems: A Survey. *ACM Computing Surveys*, 2022.

[4] G. Zheng, F. Zhang, Z. Zheng, et al. DRN: A Deep Reinforcement Learning Framework for News Recommendation. *The Web Conference*, 2018.

[5] M. Chen, A. Beutel, P. Covington, et al. Top-K Off-Policy Correction for a REINFORCE Recommender System. *WSDM*, 2019.

[6] E. Ie, V. Jain, J. Wang, et al. Reinforcement Learning for Slate-Based Recommender Systems. *arXiv:1905.12767*, 2019.

[7] A. L. Silva, D. Parra, and R. T. Icarte. On the Unexpected Effectiveness of Reinforcement Learning for Sequential Recommendation. *ICML*, 2024.

[8] C. Liu, J. Lin, J. Wang, et al. Mamba4Rec: Towards Efficient Sequential Recommendation with Selective State Space Models. *arXiv:2403.03900*, 2024.

[9] Z. Liu, Q. Liu, Y. Wang, et al. SIGMA: Selective Gated Mamba for Sequential Recommendation. *AAAI*, 2025.

[10] Y. Liang, Z. Zhang, Y. Zhu, et al. Rethinking Generative Recommender Tokenizer: Recsys-Native Encoding and Semantic Quantization Beyond LLMs. *arXiv:2602.02338*, 2026.

[11] R. Shirkavand, X. Wei, C. Wang, et al. Catalog-Native LLM: Speaking Item-ID Dialect with Less Entanglement for Recommendation. *ICLR*, 2026.

[12] Y. Hou, J. Li, X. Fu, et al. Bridging Language and Items for Retrieval and Recommendation: Benchmarking LLMs as Semantic Encoders. *arXiv:2403.03952v2*, 2026.

[13] H. Kim, J. Kim, M. Choi, S. Lee, and J. Lee. MARS: Matching Attribute-aware Representations for Text-based Sequential Recommendation. *CIKM*, 2024.

[14] Y. Liu, L. Qi, X. Mao, et al. Hyperbolic-Enhanced Mixture-of-Experts Mamba for Sequential Recommendation. *AAAI*, 2026.

[15] T. Zhu, Y. Shi, Y. Zhang, Y. Wu, F. Mo, and J.-Y. Nie. Collaboration and Transition: Distilling Item Transitions into Multi-Query Self-Attention for Sequential Recommendation. *WSDM*, 2024.

[16] R. Qiu, Z. Huang, H. Yin, and Z. Wang. Contrastive Learning for Representation Degeneration Problem in Sequential Recommendation. *WSDM*, 2022.

[17] K. Zhou, H. Yu, W. X. Zhao, and J. R. Wen. Filter-Enhanced MLP Is All You Need for Sequential Recommendation. *The Web Conference*, 2022.

[18] J. Guo, L. Du, X. Chen, et al. On Manipulating Signals of User–Item Graph: A Jacobi Polynomial-Based Graph Collaborative Filtering. *KDD*, 2023.

[19] M. Pandey. Domain-Adaptive and Scalable Dense Retrieval for Content-Based Recommendation. *arXiv:2602.00899*, 2026.

[20] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang. LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. *SIGIR*, 2020.

[21] Y. Cheng, Y. Fan, Y. Wang, and X. Li. Accurate Multi-Interest Modeling for Sequential Recommendation with Attention and Distillation Capsule Network. *Expert Systems with Applications*, vol. 243, article 122887, 2024.

[22] M. Cai, L. Chen, Y. Wang, H. Bai, P. Sun, L. Wu, M. Zhang, and M. Wang. Popularity-Aware Alignment and Contrast for Mitigating Popularity Bias. *KDD*, 2024.